

Path Planning Task Code

Explanation

The main idea

The idea that came to my mind was to make the car to walk in the middle between the yellow cone and the blue cone by making waypoints between the yellow cones and the blue cones then make the car head towards each waypoint one after the other.

How to deal with one cone with out the other color cone?

To make a ghost cone of the missing pair.

I made an assumptions that care will always move horizontally so I there is a yellow cone for example then I will check if there is a blue cone that has the same x value if not then I will create it above the cone by 2 meters.

Code walkthrough

```
path : Path2D = []  
  
if not self.cones:  
    return path
```

If there no cones will return empty path list.

```
#split the cones to 2 lists
b_cones = [cone for cone in self.cones if cone.color == 1]
y_cones = [cone for cone in self.cones if cone.color == 0]

#sorts the cones to make a conecstance output no matter the order of the cones perceived
y_cones.sort(key=lambda cone: cone.x)
b_cones.sort(key=lambda cone: cone.x)
```

First, I split the cones list into two lists based on color then sort them to be consistence no matter the output of the order of which cone perceived first.

```
#a flag if the start of the path is on left of the car or on the right
if (y_cones and y_cones[0].x < cx) or (b_cones and b_cones[0].x < cx):
    direction = -1
else:
    direction = 1
```

This flag helps the car decide whither to go left or right based on the first cone.

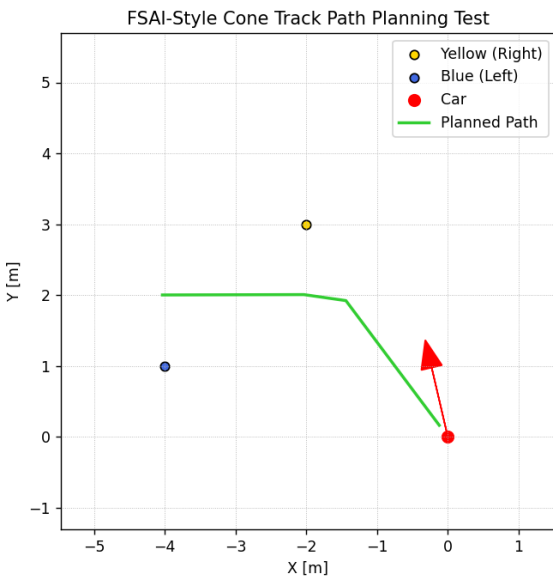


Figure 2: scenario 22

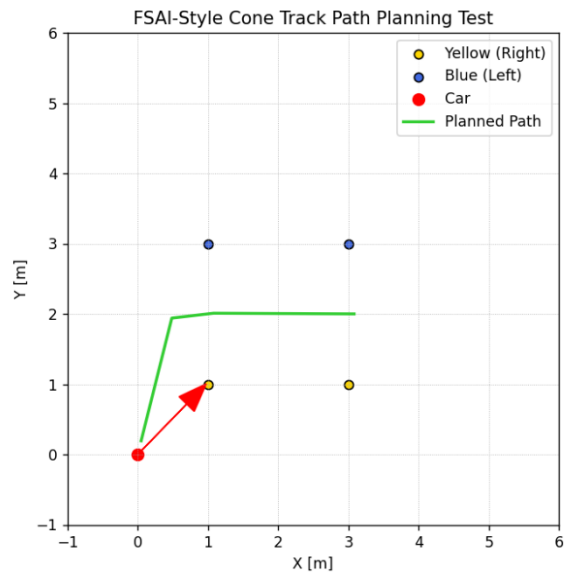


Figure 1: scenario 3

```

if direction == -1:
    y_cones.sort(key=lambda cone: cone.y)
    b_cones.sort(key=lambda cone: cone.y, reverse=True)
else:
    y_cones.sort(key=lambda cone: cone.y, reverse=True)
    b_cones.sort(key=lambda cone: cone.y)

```

This is for the edge case of more than one cone of the same color has the same x value.

Based on the direction the car will give priority to

- if direction is right (direction = 1)
 - the highest yellow cone that keeps the yellow cones to the right of the car.
 - The lowest blue cone that keeps the blue cones to the left of the car.
- if direction is left (direction = -1)
 - the lowest yellow cone that keeps the yellow cones to the right of the car.
 - The highest blue cone that keeps the blue cones to the left of the car.

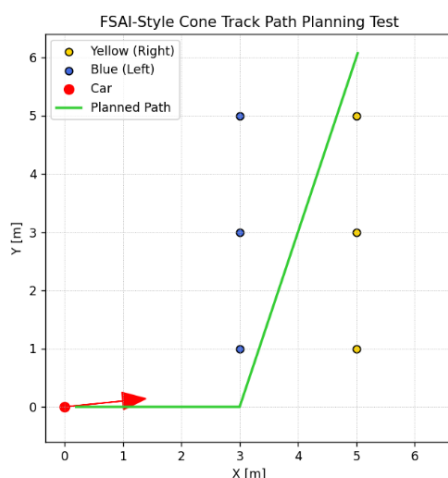


Figure 4:scenario 23

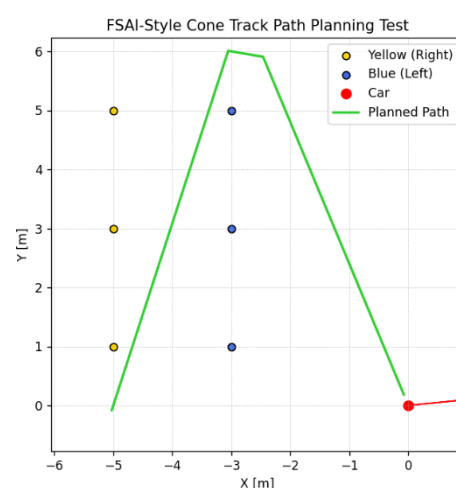


Figure 3:scenario 26

```

for y_cone in y_cones:
    seen = 0
    for b_cone in b_cones:
        if y_cone.x == b_cone.x:
            seen+=1
    if seen == 0:
        if self.car_pose.x > y_cone.x:
            b_cones.append(Cone(y_cone.x, y_cone.y-2, color=1))
        else:
            b_cones.append(Cone(y_cone.x, y_cone.y+2, color=1))

for b_cone in b_cones:
    seen = 0
    for y_cone in y_cones:
        if b_cone.x == y_cone.x:
            seen+=1
    if seen == 0:
        if self.car_pose.x > b_cone.x:
            y_cones.append(Cone(b_cone.x, b_cone.y+2, color=0))
        else:
            y_cones.append(Cone(b_cone.x, b_cone.y-2, color=0))

```

This code ensures that for every cone on one side of the track (yellow or blue), there is a matching cone on the opposite side, even if one is missing.

It does this by checking all yellow cones against blue ones (and vice versa) and adding ghost cones when needed.

```

if direction == -1:
    y_cones.sort(key=lambda cone: (cone.x, -cone.y), reverse=True)
    b_cones.sort(key=lambda cone: (cone.x, cone.y), reverse=True)
else:
    y_cones.sort(key=lambda cone: (cone.x, -cone.y))
    b_cones.sort(key=lambda cone: (cone.x, cone.y))

```

This sorts the cones so each cone has the same index in the list as its pair.

```

blue cones:
[Cone(x=1.0, y=3.0, color=1), Cone(x=3.0, y=3.0, color=1)]
yellow cones:
[Cone(x=1.0, y=1.0, color=0), Cone(x=3.0, y=1.0, color=0)]

```

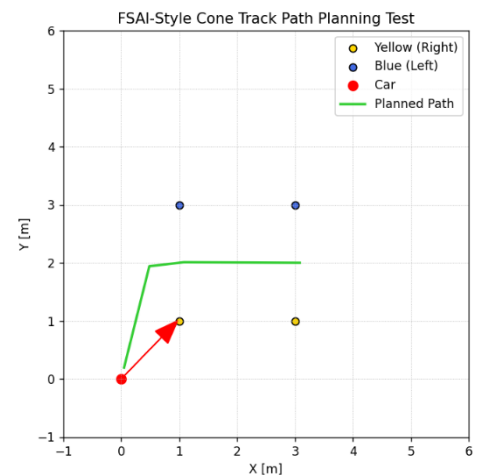


Figure 5: scenario 3

But there is an edge case where there may be one than one cone with same color on the same x value.

```

[Cone(x=3.0, y=1.0, color=1), Cone(x=3.0, y=3.0, color=1), Cone(x=3.0, y=5.0, color=1), Cone(x=5.0, y=7.0, color=1)]
yellow cones:
[Cone(x=3.0, y=-1.0, color=0), Cone(x=5.0, y=5.0, color=0), Cone(x=5.0, y=3.0, color=0), Cone(x=5.0, y=1.0, color=0)]

```

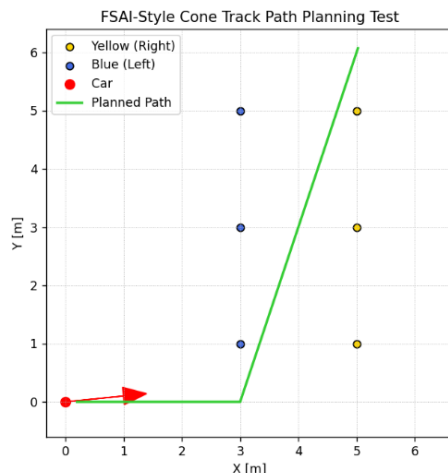


Figure 6: scenario 23

But we fix this in the waypoint assigning process.

```
waypoints : Path2D = []
waypoints.append((cx, cy))
i_b = 0
i_y = 0
for i in range(0, min(len(b_cones), len(y_cones))):
    if y_cones[i_y].x == b_cones[i_b].x:
        waypoints.append((y_cones[i_y].x, (y_cones[i_y].y + b_cones[i_b].y) / 2))
        i_b+=1
        i_y+=1
    elif direction * y_cones[i_y].x < direction * b_cones[i_b].x:
        i_y+=1
    else:
        i_b+=1
```

Here It makes a list of waypoints in the mid-point between the pairs of cones that is the first and has the same x value.

The first waypoint is the car position

```
[Cone(x=3.0, y=1.0, color=1), Cone(x=3.0, y=3.0, color=1), Cone(x=3.0, y=5.0, color=1), Cone(x=5.0, y=7.0, color=1)]
yellow cones: ✓
[Cone(x=3.0, y=-1.0, color=0), Cone(x=5.0, y=5.0, color=0), Cone(x=5.0, y=3.0, color=0), Cone(x=5.0, y=1.0, color=0)]
```

Here it looked at compared between the first cones in each list but if it doesn't find the other cone has the same x value, so it increments the blue cone index only and doesn't increment the yellow cone index until it finds a blue cone that has the same x value.

```
if direction == -1:
    waypoints.insert(1, (waypoints[1][0]+0.5, waypoints[1][1]))
else:
    waypoints.insert(1, (waypoints[1][0]-0.5, waypoints[1][1]))
```

After that we at we add waypoint just before it enters the path that deals with the problem that if the first cone is just above the car position.

After that we start making the path step by step from waypoint to another

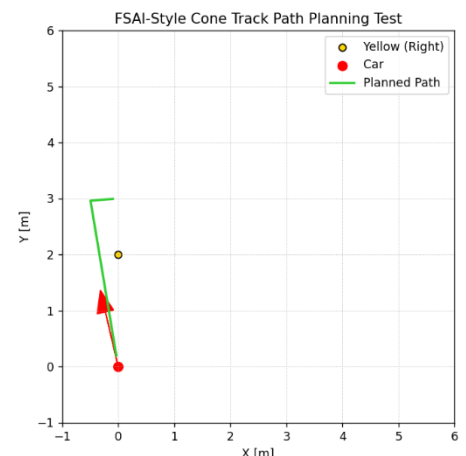


Figure 7: scenario 20

```

step = 0.2
current_x = cx
current_y = cy
car_at_wp = (cx, cy)

for j in range(1, len(waypoints)):
    dx_goal = waypoints[j][0] - current_x
    dy_goal = waypoints[j][1] - current_y

    theta_goal = math.atan2(dy_goal, dx_goal)

    dist = math.hypot(dx_goal, dy_goal)
    num_steps = round(dist/step)

    for i in range(1, num_steps+1):
        dx = math.cos(theta_goal) * step * i
        dy = math.sin(theta_goal) * step * i

        current_x = car_at_wp[0] + dx
        current_y = car_at_wp[1] + dy

        path.append((current_x, current_y))

    car_at_wp = (current_x, current_y)

return path

```

Here we get the distance between the waypoint we want to go to and the one we are in and also that angle then we calculate how many steps it will take to go this distance.

Limitation:

- The path planning account only for horizontal roads so it can't truly walk vertically but it still somewhat navigates a vertical path

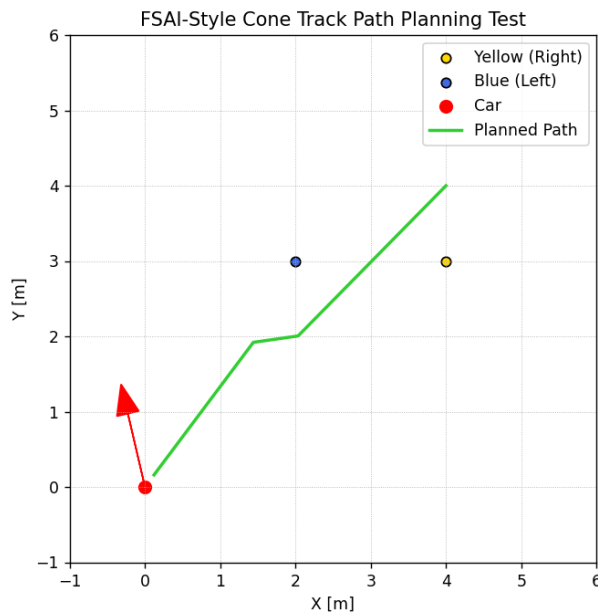


Figure 8: Scenario 24

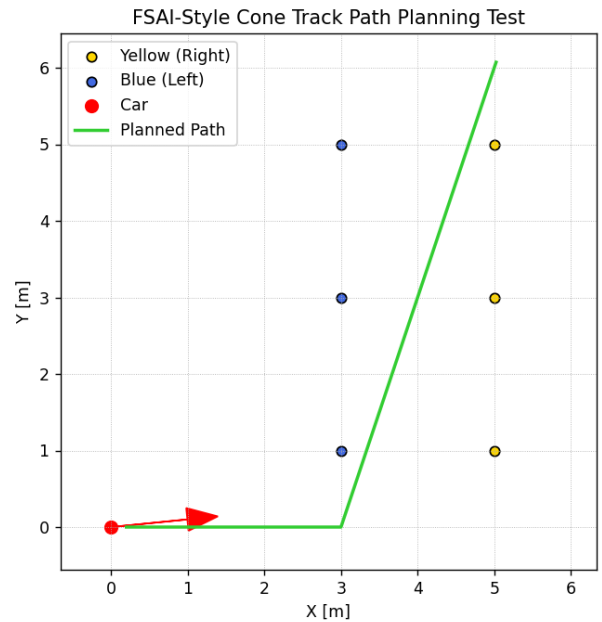


Figure 9: Scenario 23

but as we can see in scenario 23 if the vertical path is too long then the car will hit the cones in the top right corner and the bottom left corner if the car has actual dimensions.

- It can go only right or left based on the cone that is closed to it if it's on its right or on its left. That means that if a path started on the left of the car and continued towards the right the path will behave unexpectedly.

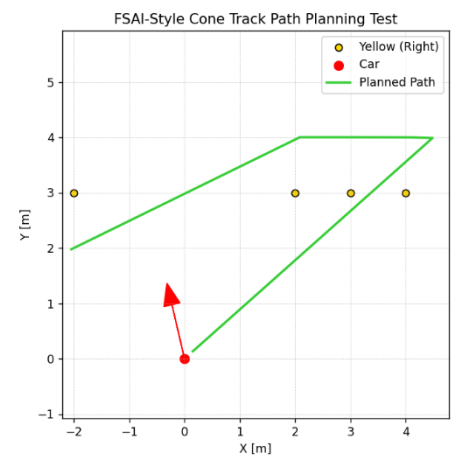


Figure 10: Scenario 25